

Original

Deleting from a data structure in $O(T(n)\log n)$

Suppose you have a data structure which allows adding elements in **true** $O(T(n))$. This article will describe a technique that allows deletion in $O(T(n)\log n)$ offline.

Algorithm

Each element lives in the data structure for some segments of time between additions and deletions. Let's build a segment tree over the queries. Each segment when some element is alive splits into $O(\log n)$ nodes of the tree. Let's put each query when we want to know something about the structure into the corresponding leaf. Now to process all queries we will run a DFS on the segment tree. When entering the node we will add all the elements that are inside this node. Then we will go further to the children of this node or answer the queries (if the node is a leaf). When leaving the node, we must undo the additions. Note that if we change the structure in $O(T(n))$ we can roll back the changes in $O(T(n))$ by keeping a stack of changes. Note that rollbacks break amortized complexity.

Notes

The idea of creating a segment tree over segments when something is alive may be used not only for data structure problems. See some problems below.

Implementation

This implementation is for the [dynamic connectivity](#) problem. It can add edges, remove edges and count the number of connected components.

```
struct dsu_save {
    int v, rnk, u, rnk_u;

    dsu_save() {}

    dsu_save(int _v, int _rnk, int _u, int _rnk_u)
        : v(_v), rnk(_rnk), u(_u), rnk_u(_rnk_u) {}
};

struct dsu_with_rollbacks {
    vector<int> p, rnk;
```